

Linguagem Imperativa 2

- Procedimentos
- Exemplos de programa
- Ambiente de execução
- Chamada de procedimento

Procedimentos

- A linguagem imperativa 2 (li2) estende a Linguagem de Imperativa 1 (li1) com procedimentos parametrizados e recursivos (de primeira ordem).
- A parte relevante da BNF da Linguagem Imperativa 2 (li2) é apresentada na Figura a seguir.

Procedimentos

Programa ::= Comando

Comando ::= "skip" | Atribuicao | IO | Comando ";" Comando
 | IfThenElse | While | **ComandoDeclaracao**
 | **ChamadaProcedimento**

ComandoDeclaracao ::= "{" Declaracao ";" Comando "}"

Declaracao ::= DeclaracaoVariavel | **DeclaracaoProcedimento**
 | DeclaracaoComposta

DeclaracaoVariavel ::= ...

DeclaracaoComposta ::= ...

DeclaracaoProcedimento ::=

 "proc" Id "(" [ListaDeclaracaoParametro] ")" "{" Comando "}"

ListaDeclaracaoParametro ::= Tipo Id | Tipo Id "," ListaDeclaracaoParametro

ChamadaProcedimento ::= "call" Id "(" [ListaExpressao] ")"

Figura 25. BNF para a Linguagem Imperativa 2

Procedimentos

- O corpo de um procedimento é um comando e a chamada de um procedimento também é um comando.
- A evolução da Linguagem Imperativa 1 (li1) para a Linguagem Imperativa 2 (li2) é análoga à
- Evolução da Linguagem de Expressão 2 (le2) para a Linguagem Funcional 1 (lf1)
 - introdução de funções versus introdução de procedimentos

Procedimentos

- Declaração de procedimento ocorre em blocos de declaração.
- Na Linguagem Imperativa 1 (li1), um bloco pode conter apenas declaração de variáveis;
- a construção é estendida aqui para permitir também declaração de procedimentos.

Procedimentos

- Um mesmo bloco pode conter os dois tipos de declaração.
- declaração de um procedimento
- chamada de um procedimento

Procedimentos

- A declaração de um procedimento inclui uma lista de identificadores (os parâmetros do procedimento) com os respectivos tipos,
- e um corpo, constituído do comando a ser executado quando o procedimento é invocado.

Procedimentos

- A chamada de um procedimento envolve o identificador do procedimento invocado
- e uma lista de expressões, que deve obedecer a ordem e os tipos dos parâmetros.

Exemplos de programa

- Um exemplo de um bloco que declara um procedimento sem parâmetros e incrementa uma variável global é dado a seguir.
- { var a = 0, proc incA () {a := a + 1};
 - call incA(); call incA(); write(a)
- }
- O valor produzido é 2, como consequência das duas chamadas ao procedimento.

Exemplos de programa

- A Linguagem Imperativa 2 implementa escopo dinâmico, como as linguagens funcionais apresentadas.
- O exemplo a seguir ilustra este fato.
- { var x = 0, proc p (int y) {x := x + y};
 - { var x = 1; call p(3); write(x) };
 - call p(4); write(x)
- }

Exemplos de programa

- O procedimento é declarado no contexto de uma variável x global ao programa.
- A primeira chamada ao procedimento ocorre no contexto de uma outra declaração da variável.
- Como o escopo é dinâmico, a primeira chamada ao procedimento considera a declaração mais interna e, portanto, o primeiro comando de escrita imprime o valor 4.

Exemplos de programa

- Na segunda chamada, apenas a declaração mais externa está em escopo e o valor produzido é o mesmo.

Exemplos de programa

- O próximo exemplo ilustra um procedimento recursivo.
- { var b = 3,
 - proc escreveRecursivo (int a) {
 - if (not (a == 0)) then {
 - var x = 0; x := a - 1;
 - write("Ola");
 - call escreveRecursivo(x)}
 - else skip
 - };
 - call escreveRecursivo(b)
- }

Exemplos de programa

- A primeira chamada ocorre como parte do corpo do procedimento e, portanto, é uma chamada recursiva.
- A segunda chamada é a invocação não recursiva do procedimento.
- O resultado produzido pelo programa é uma seqüência com três ocorrências de Ola.

Ambiente de execução

- O ambiente de execução inclui quatro componentes:
- uma pilha de mapeamentos de identificadores em valores (memória principal);
- duas listas de valores (entrada e saída);
- uma pilha de mapeamentos de identificadores em procedimentos (declarações).

Ambiente de execução

- Como na Linguagem Imperativa 1, o ambiente pode ser modificado por atribuição, leitura e por declarações de variáveis.
- A interface do ambiente é apresentada na Figura a seguir.

Ambiente de execução

```
public interface AmbienteExecucaolImperativa2 extends AmbienteExecucaolImperativa {  
    public void mapProcedimento(Id idArg, Procedimento procedimentold) throws ProcJaDeclarado;  
    public Procedimento getProcedimento(Id idArg) throws ProcNaoDeclarado;  
}
```

Figura 26. Código da interface AmbienteExecucaolImperativa2

Ambiente de execução

- Além dos métodos da interface do ambiente de execução da Linguagem Imperativa 1,
- são incluídos métodos para associar um identificador de procedimento à sua definição
 - (envolvendo os parâmetros e o comando)
- e para retornar uma definição de procedimento dado um identificador.

Chamada de procedimento

- A implementação de chamada de procedimento é semelhante à implementação de aplicação de função, visto na lf1.
- O código é apresentado na Figura a seguir.

Chamada de procedimento

```
public class ChamadaProcedimento implements Comando {
    private Id nomeProcedimento;
    private ListaExpressao parametrosReais;
    public AmbienteExecucaoImperativa executar(AmbienteExecucaoImperativa amb) {
        AmbienteExecucaoImperativa2 ambiente = (AmbienteExecucaoImperativa2) amb;
        Procedimento procedimento = ambiente.getProcedimento(nomeProcedimento);
        ambiente.incrementa();
        ListaDeclaracaoParametro parametrosFormais = procedimento
        getParametrosFormais();
        AmbienteExecucaoImperativa2 aux = bindParameters(ambiente, parametrosFormais);
        aux = (AmbienteExecucaoImperativa2) procedimento.getComando().executar(aux);
        aux.restaura();
        return aux;
    }
}
```

Figura 27. Código da classe ChamadaProcedimento

Chamada de procedimento

- Os atributos da classe são:
- o identificador do procedimento a ser chamado
- e os parâmetros da chamada (uma lista de expressões).

Chamada de procedimento

- A primeira linha do método de execução requer que o ambiente seja de fato da Linguagem Imperativa 2;
- isto permite reusar as interfaces, como explicado anteriormente.

Chamada de procedimento

- Em seguida, a definição do procedimento a ser invocado é recuperado no ambiente, a partir do identificador.

Chamada de procedimento

- O ambiente é então incrementado e um mapeamento dos identificadores dos parâmetros formais nos respectivos argumentos é incluído no ambiente.
- O comando que define o corpo do procedimento é executado neste ambiente incrementado.
- O mapeamento dos parâmetros é eliminado e o ambiente (refletindo o efeito da execução do comando) é retornado.

Atividade 1

- Usando a li2, elabore uma classe em Java chamada de **Exemplo1** para executar o seguinte comando:
- { var a = 0, proc incA () {a := a + 1};
 - call incA(); call incA(); write(a)
- }

Atividade 2

- Usando a li2, elabore uma classe em Java chamada de **Exemplo2** para executar o seguinte comando:
- { var x = 0, proc p (int y) {x := x + y};
 - { var x = 1; call p(3); write(x) };
 - call p(4); write(x)
- }

Atividade 3

- Usando a li2, elabore uma classe em Java chamada de Exemplo3 para executar o seguinte comando:
- { var b = 3,
 - proc escreveRecursivo (int a) {
 - if (not (a == 0)) then {
 - var x = 0; x := a - 1;
 - write("Ola");
 - call escreveRecursivo(x)}
 - else skip
 - };
 - call escreveRecursivo(b)
- }

Atividade 4

- Usando a li2, elabore uma classe em Java chamada de Exemplo4 para executar o seguinte comando:
- { var b = 3,
 - proc escreveRecurso (int a) {
 - if (not (a == 0)) then {
 - var x = 0; x := a - 1;
 - write("Ola");
 - call escreveRecurso(x)}
 - else skip
 - };
 - call escreveRecurso(a)
 - }

Subprojeto li2

- Converter toda a linguagem li2 que está escrita em Java para Python (inclusive com as classes de exemplos).
- Esse subprojeto completo deve ser entregue no final do curso.

Leitura recomendável

- Conceitos e Paradigmas de Programação via Projeto de Interpretadores
- Augusto Sampaio e Antônio Maranhão
- Material em preparação para o JAI-2008 (Jornadas de atualização em Informática – Sociedade Brasileira de Computação)